IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

IoT-Based Smart Home Monitoring and Intelligent Decision System Using Sensor Fusion, Finite State Machines, and Graph-Theoretic Analysis

Aayushi Priya, Abhijay M S, Bellamkonda Likhith Praveen, and Lakshmi G Ravishankar

Department of Information Science and Engineering, RV College of Engineering, Bengaluru 560059, Karnataka, India

This work was carried out under the guidance of Dr. G. S. Mamatha, Department of Information Science and Engineering, RV College of Engineering, Bengaluru. Corresponding author: Aayushi Priya (e-mail: aayushipriya.is24@rvce.edu.in).

ABSTRACT Most smart home systems on the market today still make decisions one sensor at a time: if a single reading crosses a fixed line, an alarm goes off. That approach is easy to build, but it misses the bigger picture when several conditions shift together, and it tends to either cry wolf or miss things entirely. In this paper we build and test a smart home monitoring system that looks at temperature, humidity, gas concentration, motion, and ambient light together rather than in isolation, and uses that combined picture to decide how the house is actually doing. Three ESP32 nodes collect the raw readings and publish them over MQTT to a Raspberry Pi, which calibrates the data, fuses it into a single weighted risk score, and feeds that score into a six-state finite state machine (IDLE, MONITOR, ALERT_LOW, ALERT_HIGH, CRITICAL, and FAULT) to decide what, if anything, should happen next. We also model the house itself as a directed graph of zones and run BFS, DFS, and Dijkstra's algorithm over it so the system can check connectivity and work out the safest path out during a hazard, not just the shortest one. Everything the Pi computes is pushed to a React dashboard over WebSocket for live viewing and logged to ThingSpeak for later analysis. In testing, the system classified states correctly across a range of conditions, raised alerts promptly, and gave a noticeably clearer picture of what was happening in the home than a simple threshold check would have — which is really the point of combining IoT sensing with a bit of discrete math in the first place.

INDEX TERMS Edge computing, ESP32, finite state machine, graph theory, Internet of Things, MQTT, sensor fusion, smart home automation, ThingSpeak.

I. INTRODUCTION

Walk into almost any "smart" home today and the safety logic underneath it is usually quite dumb: a gas sensor crosses a fixed number, a buzzer goes off. A temperature sensor crosses another number, another alarm fires. Each sensor is judged entirely on its own, with no sense of what the other sensors in the house are saying at the same time. That is cheap and easy to build, which is presumably why it is everywhere, but it also means the system has no way of telling a brief, harmless spike from the early stages of something genuinely dangerous. It either over-alerts on noise or stays quiet right up until a threshold is finally crossed.

This paper grew out of a fairly simple question: what if the house looked at all of its sensors together instead of one at a time? We built a monitoring system around three ESP32 nodes that track temperature, humidity, gas concentration, motion, and ambient light, and stream their readings over MQTT to a Raspberry Pi acting as an edge coordinator. Rather than reacting to any single value, the Pi fuses the readings into one risk score and feeds that score into a finite state machine, which is really just a structured way of saying "here is how worried the system should be right now, and here is what should happen next." On top of that, we treat the layout of the home as a graph, which lets the system reason about which zones are still reachable and, if something does go wrong, which way out is actually safest rather than just shortest.

In putting this together, we found ourselves contributing a few things that, as far as we could tell from the literature, do not commonly appear together:

- A sensor-fusion-plus-FSM decision layer that grades the home's condition across six states instead of collapsing everything into a binary alarm/no-alarm signal.
- A graph model of the house that supports both connectivity checking and a Dijkstra-based "safest path out" recommendation that updates as conditions change.
- A working pipeline from ESP32 nodes all the way through to a live dashboard and ThingSpeak logging, so the same data supports both instant alerts and after-the-fact analysis.
- A codebase split cleanly enough into modules that we could test the FSM transitions and the graph algorithms on their own and have some confidence they were behaving correctly.

The rest of the paper follows roughly that order. Section II looks at related work in IoT home automation. Section III lays out the system design. Section IV gets into the algorithms — sensor fusion, the FSM, and the graph analysis. Section V covers how it was actually implemented, Section VI reports what we observed when running it, and Section VII wraps up with where this could go next.

II. RELATED WORK

A fair amount of prior work has looked at MQTT-based home automation built around ESP32 boards and Node-RED dashboards for remote control [1], [3]. Chakraborty and Datta

looked at pushing computation to the edge in home automation specifically to cut latency and reduce how much the system depends on a constant cloud connection [2]. Devana et al. built an IoT system for monitoring the home and controlling appliances remotely [4], and Jerabandi and Kodabagi put together a fairly comprehensive survey of home automation architectures and the communication technologies behind them [5]. Sarmah et al.'s SURE-H system focused on a different angle entirely — securing the communication pipeline itself through stronger authentication [6]. More recently, Teja et al. described a smart-home management system with built-in fault detection and remote monitoring [7].

What stands out across these papers is that the sensing and communication side is generally solid, but the decision-making side is not. Most still judge each sensor on its own terms, and only one or two attempt anything like a graded, multi-level read of the home's overall state. None of them, as far as we found, combine sensor fusion with graph-based evacuation planning. That gap is essentially what this paper tries to fill, by tying weighted sensor fusion, a six-state FSM, and graph-based path analysis together into one edge-to-cloud system.

III. PROPOSED SYSTEM DESIGN

A. System Overview

Fig. 1 shows the overall shape of the system: a sensing layer, a communication layer, an edge-processing layer, and an application layer sitting on top of each other. Three ESP32 nodes cover different parts of the house. The environmental node carries a DHT11 and an MQ-2 and reports temperature, humidity, and gas concentration; the motion node pairs a PIR sensor with a buzzer so it can both detect movement and sound an alert; and the light node uses an LDR alongside an LED to read ambient light and give a visual indication. All three publish over MQTT to the Raspberry Pi, which is where the actual decision-making happens — fusion, FSM evaluation, health checks, and graph-based path analysis — before results are pushed out to the Node.js/React dashboard and uploaded to ThingSpeak.

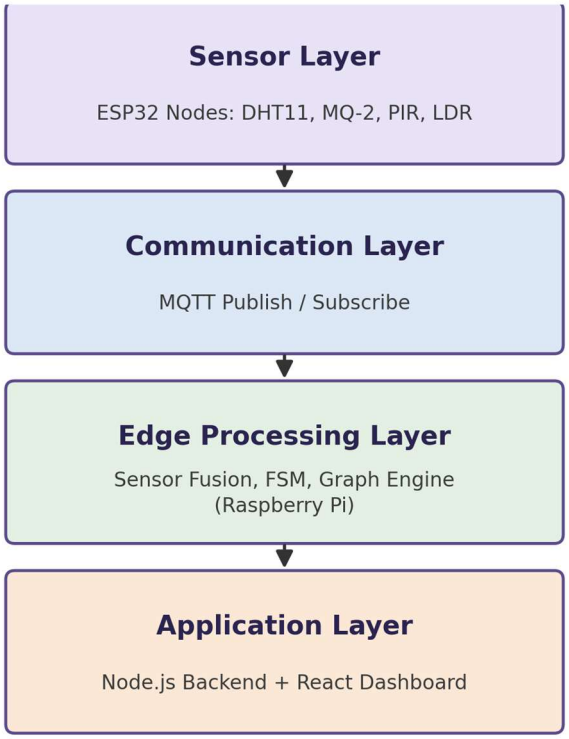

FIG. 1. Layered architecture of the proposed IoT smart home monitoring system.

B. Hardware Design

Table I lists the three nodes and what each one carries. They are all built on the same ESP32 base, which handles local acquisition and pushes readings to the broker over Wi-Fi on a regular interval.

TABLE I
SENSOR NODE SUMMARY

Node	Sensors / Actuators	Parameters Monitored
Environmental	DHT11, MQ-2	Temperature, humidity, gas concentration
Motion	PIR sensor, buzzer	Motion detection, audible alert
Light	LDR, LED	Ambient light level, visual indication

A Raspberry Pi sits at the center of all this as the edge-processing unit — it subscribes to every node's MQTT topic, pulls the incoming readings together, and runs the decision-making logic covered in Section IV.

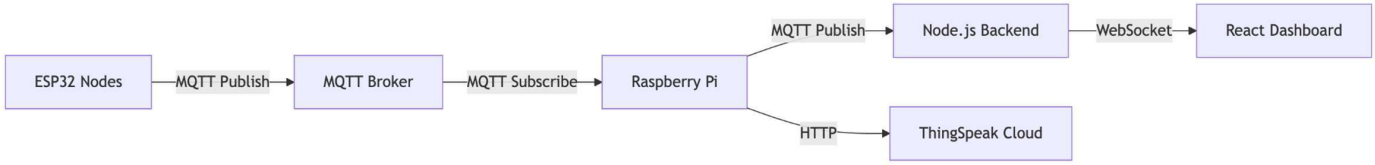

FIG. 2. Communication architecture showing MQTT publish/subscribe flow from sensor nodes to the Raspberry Pi, and onward delivery to the dashboard (WebSocket) and ThingSpeak (HTTP).

C. Communication Architecture

We chose MQTT for the messaging layer mainly because it is light enough to not be a burden on the ESP32 boards while still being reliable. The flow, shown in Fig. 2, is straightforward: each node publishes to its own topic on the broker, and the Raspberry Pi subscribes to all three so it sees everything in one place. Once it has processed a reading — risk score, FSM state, evacuation path if relevant — it republishes that result to a topic the Node.js backend listens on, which forwards it to the React dashboard over WebSocket, and it separately pushes raw and processed values to ThingSpeak over HTTP so there is a historical record to look back on. Table II lays out where each protocol fits in.

TABLE II
COMMUNICATION PROTOCOLS

Protocol	Purpose
MQTT	Communication between ESP32 nodes and the Raspberry Pi
HTTP	Uploading processed data to ThingSpeak
REST API	Historical data retrieval by the dashboard
WebSocket	Real-time dashboard updates

D. Edge Processing Layer

Most of the actual intelligence in the system lives on the Raspberry Pi. Once the raw readings arrive, it works through a handful of steps to turn them into something useful:

- Sensor calibration — corrects and normalizes the raw readings so noise and per-unit drift don't skew anything downstream.
- Sensor fusion — folds the calibrated readings from all five sensor channels into one composite risk score.
- Health monitoring — keeps an eye on each reading stream and flags anything that looks like a stuck or failing sensor.
- Finite State Machine — takes the risk score and the health status and decides which of the six operating states the system should be in.
- Graph engine — keeps a live graph of the home's zones and updates connectivity and edge weights as conditions change.
- BFS, DFS, and Dijkstra's algorithm — handle alert propagation, connectivity verification, and safest-path computation, respectively.
- Dataset recorder and ThingSpeak client — log processed readings locally and ship them to the cloud for later analysis.

E. Application Layer

On top of all that sits a Node.js backend that exposes a REST API and a WebSocket channel, feeding a React dashboard. From the dashboard you can watch sensor values update live, see the current risk score, scroll back through historical trends, follow FSM state changes as they happen, and pull up a graph view that highlights whatever evacuation path the algorithms have worked out if things turn critical.

IV. ALGORITHMIC DESIGN

A. Sensor Fusion and Risk Score Generation

The whole point of the fusion step is to stop treating each sensor as its own little island. We take temperature, humidity, gas concentration, motion, and light, normalize each reading xi to the range [0, 1], and combine them as a weighted sum:

$$R = \Sigma\, wi \cdot xi,norm\,, \qquad i = 1,\ \ldots,\ 5$$

Here wi is how much weight sensor i gets, and $R \in [0, 1]$ is the risk score that comes out the other end. We weighted the genuinely hazard-indicating signals — gas concentration especially, and any sharp temperature rise — more heavily than ambient signals like light level, since a dim room on its own says very little about danger but a gas spike says quite a lot. The result is a single number that reflects the room's overall condition rather than whichever sensor happened to trip first.

B. Finite State Machine

From there, the risk score R and the sensor health flag feed into a six-state FSM. Table III lists the states and their risk-score boundaries, and Fig. 3 shows how the system moves between them.

TABLE III
FSM STATES AND RISK THRESHOLDS

State	Risk Range	Description
IDLE	$R < 0.20$	Normal operation; all parameters within safe limits

MONITOR	$0.20 \le R < 0.40$	Minor variations requiring continued observation
ALERT_LOW	$0.40 \le R < 0.65$	Moderately abnormal; early warning issued
ALERT_HIGH	$0.65 \le R < 0.85$	Potentially hazardous; stronger alerts triggered
CRITICAL	$R \ge 0.85$	Severe condition; immediate response required
FAULT	—	Sensor malfunction or communication failure

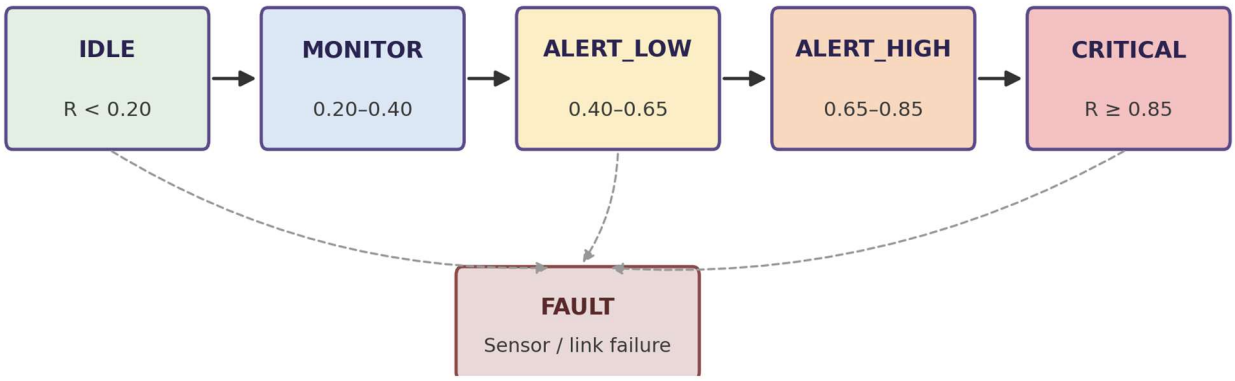

FIG. 3. Finite State Machine for system-state classification. Risk-score thresholds govern transitions between IDLE, MONITOR, ALERT_LOW, ALERT_HIGH, and CRITICAL; any state may transition to FAULT on sensor or communication failure.

C. Graph-Theoretic Analysis

Beyond the per-zone risk score, we also wanted the system to reason about the house as a whole, so we model it as a directed graph — zones, sensor nodes, and the processing/communication elements all become vertices, and each edge is weighted by how risky the connection currently is. Three classic graph algorithms do the actual work here. BFS spreads an alert outward to neighboring zones the moment a hazard shows up in one of them. DFS walks the entire graph to confirm nothing has become cut off or unreachable. And Dijkstra's algorithm finds the lowest-risk route from wherever you are to a safe exit — not necessarily the shortest one in meters, but the one that avoids the most danger along the way, recalculated as conditions shift.

V. IMPLEMENTATION

Getting this running in practice meant chaining together sensor initialization, network setup, data acquisition, fusion, FSM evaluation, alerting, graph analysis, and cloud upload into one continuous loop. On the firmware side, each ESP32 brings up its sensors and GPIO pins (DHT11, MQ-2, PIR, LDR) and connects to Wi-Fi before it starts sampling. On the Raspberry Pi, the coordinator pulls in MQTT messages as they arrive, runs calibration and fusion on them, checks the FSM, and — once the state reaches Warning or Critical — triggers the buzzer and kicks off the graph-based evacuation analysis, after which it uploads to ThingSpeak and pushes the update out to the dashboard.

We tried to keep the codebase reasonably disciplined: sensing, communication, decision logic, and the dashboard are each their own component with a clear interface to the rest. Variable names and comments are meant to be self-explanatory rather than clever, the directory structure mirrors the architecture, and common utility functions are factored out instead of copy-pasted. We also added input validation and basic fault checks throughout, since a system meant to flag hazards shouldn't itself fall over the moment a sensor misbehaves or Wi-Fi drops for a few seconds.

VI. RESULTS AND DISCUSSION

A. Execution Outcomes

Once running, the system tracked temperature, humidity, gas concentration, motion, and ambient light continuously across all three nodes, with everything showing up on ThingSpeak's dashboards for remote viewing. The FSM moved between Normal, Warning, and Critical states in line with whatever the sensors were actually reporting, and the buzzer reliably fired whenever conditions crossed into hazardous territory. The graph side held up too — the house modeled cleanly as a directed graph, and connectivity and shortest-path checks ran over it without issue.

B. Validation of Algorithm Correctness

To check the fusion logic wasn't just echoing one dominant sensor, we watched how the risk score responded to combinations of readings rather than single spikes, and it tracked the overall picture rather than any one input. For the

FSM, we deliberately varied conditions and watched the transitions: safe readings kept it in IDLE or MONITOR, moderate issues pushed it into ALERT_LOW or ALERT_HIGH, and genuinely hazardous combinations took it to CRITICAL — exactly what we'd expect if the thresholds were working as intended. On the graph side, every zone stayed reachable under connectivity checks, and Dijkstra consistently returned routes that made sense given the simulated risk levels rather than just the shortest geometric path.

C. Handling of Special Cases

A few edge cases were worth checking deliberately. Bad or out-of-range readings get filtered out before they can influence a decision. If an ESP32 loses Wi-Fi, it keeps retrying in the background rather than giving up, and resumes publishing once it reconnects. Because the fusion step smooths out short-lived fluctuations rather than reacting to every blip, and because the decision engine looks at several signals at once, something like a smoke spike that coincides with motion gets read more sensibly than it would under a plain threshold system. And if ThingSpeak is briefly unreachable, local alerting keeps working regardless — the cloud connection is a nice-to-have for history, not a dependency for safety.

D. Limitations

None of this is to say the system is finished. The fusion weights and FSM thresholds are still hand-set rather than learned from data, so they reflect our judgment more than any rigorous calibration. Sensor accuracy can drift with environmental noise over time. The graph model currently assumes the zone layout is fixed, which won't hold for a home that gets renovated or rearranged. The cloud side is only as reliable as the Wi-Fi connection. And realistically, we've only tested this on a small number of sensors and nodes, so how it behaves at the scale of a full house with many more zones remains to be seen.

VII. CONCLUSION AND FUTURE SCOPE

What we set out to build was a smart home system that thinks about its sensors together rather than separately, and that's largely what came together here — sensor fusion, a six-state FSM, and graph-based path analysis, all running on an ESP32-to-Raspberry-Pi-to-cloud pipeline backed by MQTT and ThingSpeak. Looking at multiple readings jointly through the fused risk score and the FSM gave more dependable hazard detection than a plain threshold check would have, and the graph layer added something threshold systems don't even attempt: a sense of which way is actually safe to go, not just which alarm to sound. Testing bore this out — the FSM transitioned where we expected it to, the graph algorithms behaved correctly, and both the live dashboard and the ThingSpeak history did their jobs. There's plenty of room to keep building on this: swapping the hand-tuned weights for something learned from real data, adding more sensors and nodes, building an actual mobile app instead of relying on a browser dashboard, trying smarter path-planning algorithms like A*, and eventually testing this on a larger building rather than a single simulated home.

ACKNOWLEDGMENT

We're grateful to Dr. G. S. Mamatha, Head Of Department of the Department of Information Science and Engineering, RV College of Engineering, Bengaluru, for guiding this work and pointing us in the right direction more than once along the way.

REFERENCES

[1] A. K. Arigela, C. Banapuram, and N. Venu, “Remote based home automation with MQTT: ESP32 nodes and Node-RED on Raspberry Pi,” in Proc. 8th Int. Conf. I-SMAC, 2024, pp. 313–318.

[2] T. Chakraborty and S. K. Datta, “Home automation using edge computing and Internet of Things,” in Proc. IEEE Int. Conf. Consum. Electron.–Asia (ICCE-Asia), 2017, pp. 47–50.

[3] P. Chouragadey, B. Rathore, and G. Khare, “Smart home automation with MQTT using ESP32 and Node-RED,” in Proc. 4th Int. Conf. Power, Control Comput. Technol. (ICPC2T), 2025.

[4] V. N. K. R. Devana, P. S. S. Pentapati, S. H. Medisetti, K. S. S. Parvathini, S. Siddanthupu, and V. V. R. Karna, “IoT based home monitoring and appliance control system,” in Proc. Int. Conf. Eng. Innov. Technol. (ICoEIT), 2025.

[5] M. Jerabandi and M. M. Kodabagi, “A review on home automation system,” in Proc. Int. Conf. Intell. Comput., Instrum. Control Technol. (ICICICT), 2017, pp. 1411–1415.

[6] R. Sarmah, M. Bhuyan, and M. H. Bhuyan, “SURE-H: A secure IoT enabled smart home system,” in Proc. IEEE 5th World Forum Internet of Things (WF-IoT), 2019, pp. 59–63.

[7] D. C. Teja, M. B. N. Rao, and D. Gokulakrishnan, “IoT-enabled smart home management system featuring fault detection and remote monitoring,” in Proc. Int. Conf. Comput. Commun. Technol. (ICCCT), 2025.